

AERIS

AI-ASSISTED AUTONOMOUS DISASTER RESPONSE & EMERGENCY LOGISTICS UAV

Software Technical Stack, End-to-End Pipeline, Algorithms, Mathematics, Runtime Architecture & Implementation Plan

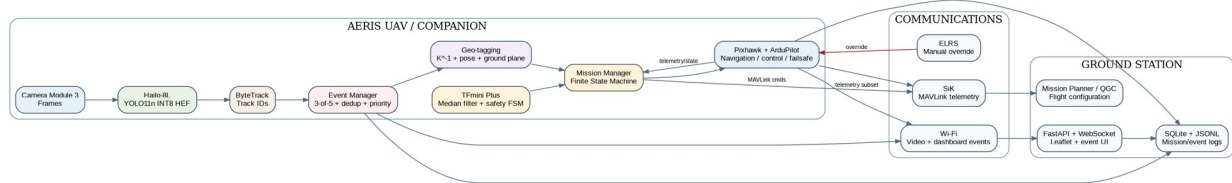


Figure 1. Canonical AERIS software/hardware/ground architecture.

Revision 1.0 | 26 September 2026

Purpose: mentor review, technical refinement, implementation baseline and build execution.

Engineering status

This document treats the supplied AERIS Rev 2.0 master build specification as the project baseline and expands its software architecture into an explicit input -> processing -> output -> destination contract. Items explicitly marked Proposed / Engineering Addition are additions intended to make implementation and mentor review more precise; they are not claims that the existing project files had already frozen those choices.

0. Document Control and Reading Guide

Field	Value
Project	AERIS - AI-Assisted Autonomous Disaster Response & Emergency Logistics UAV
Baseline source	AERIS Prototype Master Build Specification Rev 2.0, 26 September 2026
This document	Software-focused expansion of the Rev 2.0 baseline
Prototype	F450-class quadcopter; 5-kg hexacopter is digital-twin target only
Primary companion	Raspberry Pi 5 4 GB + Raspberry Pi AI HAT+ / Hailo-8L
Primary AI	Custom YOLO11n detector -> Hailo HEF; person / vehicle / debris baseline
Tracking	ByteTrack
Mission logic	Deterministic finite-state machine + explicit safety interlocks
Flight authority	Pixhawk + ArduPilot Copter
Telemetry	MAVLink over Pi <-> Pixhawk and SiK to GCS
Operator link	ELRS manual override; not interchangeable with SiK
Ground UI	Mission Planner / QGroundControl + local FastAPI/WebSocket dashboard
Data	SQLite + JSONL + Pixhawk flight log
Simulation	Gazebo + ArduPilot SITL

0.1 What this document answers

- What software components exist, where they run, and why they are selected.
- What input enters each stage, what processing happens inside the stage, what output is produced, and where that output goes.
- Which model, algorithm, logic, mathematical transformation, optimizer, loss/error metric, filter, or control method is used.
- What remains a tunable parameter versus what is frozen by the current project baseline.
- What software files will be created, how they communicate, and how the system is built and tested in a safe order.

0.2 Source boundary

The supplied Rev 2.0 specification already freezes the main software stack, model family, tracking, geo-tagging method, obstacle behavior, mission state machine, runtime cadences, repository structure and validation gates. This document preserves those choices and adds implementation-level contracts, explicit training/error terminology, typed interfaces and development guidance where the source specification was

intentionally high-level. The physical prototype remains a research/hackathon system and not a production search-and-rescue aircraft.

1. System Objective and Core Working

AERIS converts aerial observations into structured incident information and connects that information to supervised response behavior. The canonical loop is: mission definition -> autonomous takeoff -> coverage search -> camera capture -> edge AI -> detection -> tracking/confirmation -> geo-location -> priority -> ground alert -> human verification -> response waypoint -> supervised payload release (optional) -> return or resume search.



Figure 2. End-to-end data and command flow.

Key architectural rule

The Raspberry Pi is the companion computer and intelligence layer; Pixhawk/ArduPilot remains the flight-control and failsafe authority. The AI can propose an incident and the mission manager can request high-level actions, but the flight controller owns low-level stabilization, navigation, actuator control and stronger failsafes.

2. Canonical End-to-End Pipeline Contract

Stage	Input	Source	Inside the stage	Output	Output destination
P0	Mission / configuration input	Operator, mission YAML, GCS	Load polygon/target, thresholds, reserves, geofence and release policy	MissionConfig	Mission Manager / PRECHECK
P1	Camera capture	Camera Module 3 / replay video	Timestamp, frame ID, maintain latest-frame buffer	FramePacket	Preprocess / Hailo
P2	Preprocessing	FramePacket	Resize/crop/color transform to deployment input	TensorFrame	Hailo inference
P3	AI inference	TensorFrame	YOLO11n INT8 HEF on Hailo-8L	RawDetectionBatch	Postprocess
P4	Postprocessing	RawDetectionBatch	Confidence threshold + NMS / decoder output normalization	DetectionBatch	ByteTrack
P5	Tracking	DetectionBatch + prior tracks	Associate detections, preserve IDs	TrackBatch	Event manager
P6	Temporal confirmation	TrackBatch history	3-of-5 confirmation + cooldown/spatial suppression	ConfirmedTrack	Incident builder

Stage	Input	Source	Inside the stage	Output	Output destination
P7	Telemetry synchronization	ConfirmedTrack + VehicleState cache	Nearest valid timestamp association within configured sync tolerance	SynchronizedObservation	Geo-tag
P8	Geo-tagging	Pixel/box + camera K + extrinsics + pose + altitude + ground estimate	Ray transform + ground intersection	GeoEstimate	Incident builder
P9	Priority	confidence + persistence + class + context	Weighted transparent score	PriorityDecision	Dashboard + Mission Manager
P10	Incident logging	All event fields	Write mission/event records atomically	IncidentEvent	SQLite/JSONL/ dashboard
P11	Operator review	IncidentEvent + live video/map	Human verify / ignore / authorize response	OperatorDecision	Mission Manager
P12	Mission decision	Mission state + VehicleState + safety states + OperatorDecision	FSM transition + interlocks	MissionCommand	MAVLink command layer
P13	Obstacle safety	TFmini samples + velocity/state	Median filter + warning/hold state machine	ObstacleState	Mission Manager
P14	Flight execution	MissionCommand + ArduPilot mode	Autonomous navigation / stabilization / actuator output	VehicleState	Telemetry + flight log
P15	Payload authorization	OperatorDecision + safety interlocks + payload state	Release gate; no AI direct release	ReleaseCommand	MAVLink -> Pixhawk AUX/gripper
P16	Return/resume/failsafe	Mission completion or fault	Return home, resume context, abort or configured failsafe	FinalMissionState	GCS + logs

3. Detailed Pipeline - Input to Output at Every Stage

3.1. Mission and Configuration Intake

Converts an operator-defined mission and versioned configuration into a validated machine-readable runtime configuration.

Contract field	Definition
Input(s)	Mission polygon or target coordinate; mission.yaml; ai.yaml; sensor.yaml; network.yaml; geofence; battery reserve; payload policy; vehicle telemetry.
Processing inside	Parse schema; validate ranges; check geofence containment; compute expected route footprint; load model/version metadata; initialize mission context and mission_id.

Contract field	Definition
Why selected / role	Deterministic validation keeps mission behavior reproducible and testable; configuration is separated from code so field tuning does not require source changes.
Model / algorithm / logic	Deterministic validation keeps mission behavior reproducible and testable
Mathematics / transformation	Schema validation + deterministic configuration logic; finite-state machine entry to PRECHECK.
Output(s)	Validated MissionConfig + MissionContext + mission_id.
Where output goes	Mission Manager, route generator, logger.
Failure / degraded behavior	Reject invalid configuration; remain in PRECHECK; never start autonomous response from malformed or stale configuration.
Metrics / validation	Configuration validation pass/fail; mission initialization latency; invalid-config test coverage.

Engineering note

Freeze configuration hashes with every mission release.

3.2. Camera Capture

Acquire repeatable image frames with timestamps so downstream perception can be synchronized with vehicle state.

Contract field	Definition
Input(s)	Camera Module 3 via Picamera2/rpicam; camera settings; system clock.
Processing inside	Capture frames; attach frame_id and timestamp; maintain latest-frame ring buffer; drop stale frames rather than building unbounded latency.
Why selected / role	The source spec freezes Picamera2/rpicam and 15-20 FPS as an initial target. A latest-frame policy favors fresh observations over processing an old backlog.
Model / algorithm / logic	The source spec freezes Picamera2/rpicam and 15-20 FPS as an initial target. A latest-frame policy favors fresh observations over processing an old backlog.
Mathematics / transformation	Producer-consumer buffer; monotonic timestamps; bounded queue.
Output(s)	FramePacket = {frame_id, t_capture, image, width, height, camera_config_id}.
Where output goes	Preprocess worker and diagnostic recorder.
Failure / degraded behavior	Camera disconnect, timeout, invalid frame -> set camera_health false; disable AI events; flight controller remains independent.

Contract field	Definition
Metrics / validation	FPS; frame age; dropped-frame count; capture jitter; camera health heartbeat.

Engineering note

Video sources for laptop/replay should implement the same FramePacket contract.

3.3. Image Preprocessing

Convert camera frames to the model deployment representation while preserving the geometry needed for interpretation.

Contract field	Definition
Input(s)	FramePacket; model input size 640x640; camera calibration.
Processing inside	Resize/letterbox or fixed crop according to the final deployment preprocessing; color conversion; normalize exactly as required by the exported model. Preserve scale/padding metadata for box back-projection.
Why selected / role	A fixed 640x640 input is frozen in Rev 2.0 and current Hailo export is fixed-shape.
Model / algorithm / logic	A fixed 640x640 input is frozen in Rev 2.0 and current Hailo export is fixed-shape.
Mathematics / transformation	Geometric scaling; coordinate transform from model coordinates to native pixels.
Output(s)	TensorFrame + PreprocessMeta(scale, pad, source_dims).
Where output goes	Hailo inference; metadata consumed by postprocessing.
Failure / degraded behavior	Bad dimensions, preprocessing mismatch, or metadata loss -> reject frame and increment pipeline error counter.
Metrics / validation	Preprocess latency p50/p95; coordinate round-trip tests; zero-copy/copy counts.

Engineering note

The preprocessing chain must be identical between benchmark and flight builds.

3.4. YOLO11n Edge Inference

Detect the primary incident classes on the onboard accelerator.

Contract field	Definition
Input(s)	TensorFrame 640x640; YOLO11n HEF; HailoRT.

Contract field	Definition
Processing inside	Submit tensor to Hailo; run INT8 inference; read detector outputs; attach inference timestamp and model release ID.
Why selected / role	YOLO11n is the small edge baseline frozen by Rev 2.0; the current Ultralytics Hailo guide documents YOLO11 Hailo export to Hailo-8L and fixed input shape.
Model / algorithm / logic	Deep neural object detection; INT8 quantized inference via HailoRT.
Mathematics / transformation	Bounding-box regression + class confidence; internally the YOLO11 family includes DFL for box localization.
Output(s)	RawDetectionBatch.
Where output goes	Postprocessing / decoder normalization.
Failure / degraded behavior	No inference heartbeat, Hailo failure, thermal issue -> mark AI unhealthy; no false healthy status; mission policy decides whether search can continue.
Metrics / validation	Inference latency p50/p95; FPS; NPU utilization; temperature; confidence distribution; model checksum.

Engineering note

The deployed HEF is a build artifact. Do not treat a generic yolo11n.pt file as the flight model.

3.5. Postprocessing and NMS

Convert raw detector outputs into clean candidate detections.

Contract field	Definition
Input(s)	RawDetectionBatch; confidence threshold 0.35 baseline; NMS IoU 0.45 baseline.
Processing inside	Decode boxes/classes; filter by confidence; remove redundant overlapping boxes; map coordinates back to source image.
Why selected / role	NMS is appropriate for conventional one-to-many detector outputs; the source baseline uses 0.45 IoU as a starting value.
Model / algorithm / logic	Confidence thresholding + IoU-based Non-Maximum Suppression.
Mathematics / transformation	$\text{IoU} = \text{intersection_area} / \text{union_area}$. Candidate retained when $\text{confidence} \geq \text{threshold}$; overlapping boxes above NMS IoU are suppressed according to detector postprocessing.
Output(s)	DetectionBatch{class,confidence,bbox,frame_id,timestamp}.
Where output goes	ByteTrack.
Failure / degraded behavior	Malformed boxes, NaN, impossible coordinates -> discard and increment validation error.

Contract field	Definition
Metrics / validation	Precision, recall, mAP50, mAP50-95 on held-out test set; false positives/frame.

Engineering note

The release threshold must be measured on the actual model and scene set, not copied from a generic tutorial.

3.6. Multi-Object Tracking with ByteTrack

Maintain target identity across frames so the event layer can distinguish persistence from single-frame noise.

Contract field	Definition
Input(s)	DetectionBatch + track history + timestamps.
Processing inside	Associate detections to active tracks; update track state; create new tracks; age/drop stale tracks.
Why selected / role	ByteTrack is frozen in Rev 2.0 and is operationally useful because stable track IDs enable temporal confirmation and de-duplication.
Model / algorithm / logic	ByteTrack; association cost based on spatial/score consistency in the tracker implementation.
Mathematics / transformation	Association can be viewed as minimizing an assignment cost C_{ij} between observations and predicted/current tracks. The exact internal matching is delegated to the selected implementation.
Output(s)	TrackBatch{track_id,class,bbox,confidence,age,history}.
Where output goes	Event manager / temporal confirmation.
Failure / degraded behavior	Tracker reset or stale timestamps -> close tracks; never combine unrelated tracks solely by proximity.
Metrics / validation	Track fragmentation; ID switches; track lifetime; IDF1/HOTA as optional advanced evaluation.

Engineering note

Tracking metrics are an engineering addition; the source spec primarily requires stable IDs and event de-duplication.

3.7. Temporal Confirmation and Event De-duplication

Turn candidate tracks into explicit incident candidates while reducing one-frame false alarms and repeated alerts.

Contract field	Definition
Input(s)	TrackBatch history; detector confidence; configured confirmation window; cooldown; spatial cell.
Processing inside	Require detections in ≥ 3 of the last 5 inference frames; apply event cooldown and spatial suppression; create one incident per confirmed track unless policy permits another.
Why selected / role	The 3-of-5 rule and cooldown are frozen starting values in Rev 2.0 because they are transparent, deterministic and easy to validate.
Model / algorithm / logic	Finite-window persistence rule + cooldown + spatial-cell de-duplication.
Mathematics / transformation	Confirmation condition: $\text{count_positive}(\text{track}, \text{last_5_frames}) \geq 3$.
Output(s)	ConfirmedTrack / IncidentCandidate.
Where output goes	Incident builder / geo-tagging.
Failure / degraded behavior	Intermittent observations -> keep track but delay incident; track loss -> no autonomous response; log uncertainty.
Metrics / validation	Confirmation delay; false alert rate; missed-confirmation rate; duplicate incidents per run.

Engineering note

Tune from recorded field sequences; do not optimize thresholds on the same scenes used for final reporting.

3.8. Telemetry Synchronization

Associate a detection timestamp with the closest valid vehicle state before geo-tagging.

Contract field	Definition
Input(s)	ConfirmedTrack timestamp; VehicleState cache containing GPS, altitude, attitude, mode, health; configurable sync tolerance.
Processing inside	Search telemetry cache for nearest timestamp; accept only samples inside the synchronization tolerance; otherwise mark association invalid and use defined fallback policy.
Why selected / role	Nearest-timestamp association is simple, deterministic and directly aligned with the baseline; the tolerance makes the rule explicit.
Model / algorithm / logic	Nearest-timestamp association is simple, deterministic and directly aligned with the baseline
Mathematics / transformation	Nearest-neighbor temporal association: choose $\text{argmin} t_{\text{vehicle}} - t_{\text{detection}} $ subject to $ \Delta t \leq \Delta t_{\text{sync}}$.
Output(s)	SynchronizedObservation{detection, t_{vehicle} , lat, lon, altitude, attitude, age}.
Where output goes	Geo-tagging and event metadata.

Contract field	Definition
Failure / degraded behavior	No sample within tolerance -> no valid association; record association_status=INVALID; never silently pretend the aircraft pose is synchronized.
Metrics / validation	Time offset ms; association rejection rate; telemetry age; clock drift.

Engineering note

This is an engineering clarification of the baseline nearest-timestamp rule.

3.9. Camera Geometry and Approximate Geo-tagging

Estimate the target ground position instead of incorrectly treating aircraft GPS as target GPS.

Contract field	Definition
Input(s)	Detection pixel (u,v); camera intrinsics K; camera-to-body rotation R_cb; body-to-world rotation R_bw; camera position C; estimated ground height z_ground.
Processing inside	Convert image point to camera ray; transform ray into world coordinates; intersect with local ground plane; transform world offset to geographic coordinates. Validate ray direction and conditioning.
Why selected / role	This method uses available camera geometry + vehicle pose and is explicitly described as approximate in Rev 2.0.
Model / algorithm / logic	This method uses available camera geometry + vehicle pose and is explicitly described as approximate in Rev 2.0.
Mathematics / transformation	$r_c = K^{-1}[u,v,1]^T$; $r_w = R_{bw} R_{cb} r_c$; $\lambda = (z_{ground} - C_z)/r_{w,z}$; $P = C + \lambda r_w$.
Output(s)	GeoEstimate{lat,lon,quality,error_status,range_m}.
Where output goes	Incident builder; dashboard; mission manager only when quality is acceptable.
Failure / degraded behavior	Invalid ray/horizon, missing pose, bad calibration -> fallback to aircraft GPS with geo_quality=APPROXIMATE and target-navigation lockout if geometry is invalid.
Metrics / validation	Median geo error; 95th percentile error; bias versus surveyed markers; invalid-geometry rate.

Engineering note

Acceptance must use surveyed markers. The illustrative 2.4 m RSS number in Rev 2.0 is not a guarantee.

3.10. Priority Scoring

Convert a confirmed incident into transparent operational triage information for the operator.

Contract field	Definition
Input(s)	confidence c; temporal persistence t; class severity s; mission context m.
Processing inside	Normalize terms to [0,1]; calculate weighted score; map score to priority band.
Why selected / role	A transparent weighted rule was chosen instead of a black-box learned triage model so the operator can inspect and tune the logic.
Model / algorithm / logic	A transparent weighted rule was chosen instead of a black-box learned triage model so the operator can inspect and tune the logic.
Mathematics / transformation	$\text{priority_score} = 100 \cdot (0.55c + 0.20t + 0.15s + 0.10m)$. Baseline class severity: person 1.00, vehicle 0.70, debris 0.50. Bands: ≥ 80 CRITICAL, 60-79 HIGH, 40-59 NORMAL, < 40 LOG ONLY.
Output(s)	PriorityDecision{score,band,explanation}.
Where output goes	Dashboard and mission manager.
Failure / degraded behavior	Unknown class/context -> conservative default, do not escalate; configuration checksum recorded.
Metrics / validation	Priority stability; operator override frequency; false escalation rate; event-to-review latency.

Engineering note

The weights and class severities are starting values and must be validated with real mission traces.

3.11. Incident Record and Persistent Logging

Create a durable, auditable record that survives UI restarts and supports replay.

Contract field	Definition
Input(s)	IncidentEvent fields; mission_id; software/model/config release IDs.
Processing inside	Validate schema; write SQLite row; append JSONL record; flush after confirmed events; publish dashboard event.
Why selected / role	SQLite gives queryable local state; JSONL gives simple append-only interchange; Pixhawk flight log remains a separate source-of-truth flight artifact.
Model / algorithm / logic	SQLite gives queryable local state
Mathematics / transformation	Structured serialization; transaction/flush semantics; immutable mission IDs.
Output(s)	IncidentEvent with timestamp_utc, event_id, class, confidence, track_id, drone GPS, target GPS, altitude, priority, geo_quality, mode, operator_status.
Where output goes	SQLite/JSONL; dashboard; post-flight analysis.

Contract field	Definition
Failure / degraded behavior	Disk full, schema error, write failure -> raise health alarm; never silently drop confirmed incidents; keep in-memory retry queue within bounded size.
Metrics / validation	Write latency; log completeness; dropped-event count; mission header consistency.

Engineering note

UTC is the canonical persisted time; UI may render local time.

3.12. Ground Dashboard and Operator Review

Present events and mission context to a human without becoming the flight controller.

Contract field	Definition
Input(s)	WebSocket event stream; telemetry subset; video stream; map tiles; event records.
Processing inside	FastAPI API; WebSocket pushes; Leaflet/OpenStreetMap map; live video; event list/detail; operator decision controls.
Why selected / role	Local-first architecture minimizes external dependencies during a field demo and keeps authorization path explicit.
Model / algorithm / logic	FastAPI + WebSocket + Leaflet; video transport selected at integration (MJPEG/RTP/WebRTC).
Mathematics / transformation	Event ordering by timestamp/sequence; stale-state timer; UI state machine mirrors mission status but does not own flight-control state.
Output(s)	OperatorDecision and UI state; rendered event markers; audit log.
Where output goes	Mission Manager via validated command/authorization interface.
Failure / degraded behavior	Dashboard crash/network loss -> Pi/flight system continues according to configured mission/failsafe; no direct servo actuation from UI.
Metrics / validation	WebSocket latency; stale indicator; operator action acknowledgement; UI event ordering.

Engineering note

Keep dashboard privileges narrow: no raw PWM/servo pulse endpoint.

3.13. Mission State Machine

Convert perception, operator decisions and vehicle/safety conditions into deterministic mission behavior.

Contract field	Definition
Input(s)	MissionContext; VehicleState; ObstacleState; IncidentEvent; OperatorDecision.
Processing inside	Evaluate guard conditions; execute state transitions; emit allow-listed high-level commands; log every transition with reason.
Why selected / role	A finite-state machine is easier to test, replay and audit than an opaque learned policy.
Model / algorithm / logic	A finite-state machine is easier to test, replay and audit than an opaque learned policy.
Mathematics / transformation	States: IDLE -> PRECHECK -> TAKEOFF -> TRANSIT -> SEARCH -> DETECT_CONFIRM -> GEO_TAG -> PRIORITIZE -> ALERT -> WAIT_OPERATOR -> RESPONSE -> ARRIVAL -> DELIVERY_AUTH -> RELEASE -> RETURN_RESUME; ABORT from critical paths.
Output(s)	MissionCommand{type,args,reason,issued_at,mission_id}.
Where output goes	MAVLink command layer + logger.
Failure / degraded behavior	Critical fault -> ABORT; release gate closed; stronger flight-controller failsafe takes precedence.
Metrics / validation	Transition coverage; illegal transition count; command acknowledgement rate; stuck-state timeout.

Engineering note

Safety precedence is: CRITICAL VEHICLE SAFETY > RC OVERRIDE > FLIGHT CONTROLLER FAILSAFE > MISSION ABORT > AI DECISION.

3.14. Obstacle Distance Filtering

Convert TFmini range samples into a conservative obstacle state.

Contract field	Definition
Input(s)	TFmini distance samples d_k ; vehicle state; configured thresholds.
Processing inside	Reject invalid/out-of-range data; median-filter a 5-sample window; classify NORMAL/WARNING/HOLD.
Why selected / role	One forward TFmini cannot provide 360-degree perception, so Rev 2.0 deliberately freezes conservative warning/hold behavior instead of claiming full autonomous obstacle avoidance.
Model / algorithm / logic	One forward TFmini cannot provide 360-degree perception, so Rev 2.0 deliberately freezes conservative warning/hold behavior instead of claiming full autonomous obstacle avoidance.
Mathematics / transformation	Filtered distance = $\text{median}(d_{\{k-2\}}, \dots, d_{\{k+2\}})$; NORMAL if $d > 8.0\text{m}$; WARNING if $6.5 < d \leq 8.0\text{m}$; HOLD if $d \leq 6.5\text{m}$.
Output(s)	ObstacleState{distance,quality,state}.
Where output goes	Mission Manager / safety guard.

Contract field	Definition
Failure / degraded behavior	Stale/invalid sensor -> conservative state as policy requires; no reroute claim; operator can inspect.
Metrics / validation	False-stop rate; stale-sample rate; detection distance error; time-to-hold from threshold crossing.

Engineering note

Thresholds are initial values and must be tuned using measured braking/hold behavior.

3.15. MAVLink Command and Telemetry Layer

Provide a narrow, testable bridge between the companion computer and Pixhawk.

Contract field	Definition
Input(s)	MissionCommand; Pixhawk MAVLink stream; vehicle telemetry; heartbeat.
Processing inside	Read heartbeat; decode required messages; maintain VehicleState cache; send allow-listed commands; detect stale link; stop command sending when the cache is stale.
Why selected / role	pymavlink is the primary implementation in the Rev 2.0 software procedure because it gives direct MAVLink access with a small abstraction layer.
Model / algorithm / logic	pymavlink is the primary implementation in the Rev 2.0 software procedure because it gives direct MAVLink access with a small abstraction layer.
Mathematics / transformation	MAVLink serialization; heartbeat freshness; command allow-list; acknowledgement handling.
Output(s)	VehicleState packets; CommandAck status; link_health; age_ms.
Where output goes	Mission Manager; logger; dashboard telemetry.
Failure / degraded behavior	Stale MAVLink or missing heartbeat -> command gate closes; mission abort policy; flight controller keeps control/failsafe.
Metrics / validation	Heartbeat age; packet rate; command ACK rate; stale-link duration.

Engineering note

The companion link is not the same thing as the SiK-to-GCS telemetry link; interfaces must be kept distinct.

3.16. ArduPilot Flight Execution

Execute navigation, stabilization and safety behavior independently of the AI process.

Contract field	Definition
Input(s)	Waypoints, mode commands, vehicle sensors, RC input, battery, EKF/GPS.
Processing inside	ArduPilot controls attitude/position loops, mission execution, actuator outputs, battery/GCS/RC/EKF failsafes.
Why selected / role	The project explicitly does not implement a custom flight controller. Keeping flight control in ArduPilot separates safety-critical control from best-effort AI.
Model / algorithm / logic	The project explicitly does not implement a custom flight controller. Keeping flight control in ArduPilot separates safety-critical control from best-effort AI.
Mathematics / transformation	Feedback control; PID-family control loops in the flight controller; mission/position control; configured failsafe state machines.
Output(s)	VehicleState + actuator outputs + flight log.
Where output goes	Telemetry cache; physical ESCs/motors; GCS.
Failure / degraded behavior	RC loss, battery low/critical, GPS/EKF, GCS telemetry loss or other configured failsafe -> controller executes its configured safe action; Pi cannot veto a stronger controller failsafe.
Metrics / validation	Hover stability; position error; mode transition success; failsafe response time.

Engineering note

Current ArduPilot documentation describes companion computers as MAVLink-connected Linux systems that can use autopilot telemetry for intelligent decisions, while GCS failsafe is based on MAVLink heartbeat loss and configurable action.

3.17. Response Authorization and Payload Release

Ensure consequential payload release is human-authorized and interlocked.

Contract field	Definition
Input(s)	Operator authorization; mission state; current mode; battery reserve; GPS/EKF health; TFmini state; payload state.
Processing inside	Evaluate all interlocks; create ReleaseCommand only if every required guard passes; send through MAVLink to Pixhawk gripper/servo function; verify feedback when available.
Why selected / role	The source specification explicitly prohibits AI-only release and random dashboard clicks from directly pulsing the servo.
Model / algorithm / logic	AND-gated authorization logic; state transition DELIVERY_AUTH -> RELEASE.
Mathematics / transformation	release_allowed = operator_authorized AND mode_ok AND battery_ok AND gps_ekf_ok AND obstacle_ok AND payload_ready AND mission_context_ok.

Contract field	Definition
Output(s)	ReleaseCommand + release_result.
Where output goes	MAVLink -> Pixhawk -> servo/gripper; log.
Failure / degraded behavior	Any guard false -> release rejected; no retry loop without explicit new authorization. Servo jam -> abort release and return/hold per policy.
Metrics / validation	False-release count (must be zero); rejected-release reason coverage; release acknowledgement.

Engineering note

Physical payload remains a small demonstration payload until measured thrust margin and flight validation pass.

3.18. Return, Resume and Mission Completion

Close the mission safely while preserving context for re-entry.

Contract field	Definition
Input(s)	Mission completion, operator decision, failsafe state, home point, last search segment.
Processing inside	Choose RETURN_RESUME outcome; request RTL/return or continue from saved route context; flush logs; persist final status.
Why selected / role	Separating return/resume as an explicit state makes mission continuity and abort behavior testable.
Model / algorithm / logic	Separating return/resume as an explicit state makes mission continuity and abort behavior testable.
Mathematics / transformation	Deterministic state transition + route index checkpointing.
Output(s)	FinalMissionState + mission summary.
Where output goes	GCS, logs, dashboard, replay artifacts.
Failure / degraded behavior	Repeated mission aborts or inconsistent state -> remain in safe return/hold mode; no payload action.
Metrics / validation	Return success; resume continuity; mission completion latency; log flush completeness.

Engineering note

Post-flight analysis uses common mission_id across companion and Pixhawk logs.

4. Model Development, Training, Optimization and Error Handling

The physical detector is not “YOLO11n out of the box”. The engineering target is a custom-trained YOLO11n model that is exported to a Hailo-8L-compatible HEF and then evaluated again on the actual deployment pipeline. The first physical claim set is limited to person, vehicle and debris.



Figure 3. Detailed perception pipeline.

4.1 Dataset pipeline

Step	Input	Processing	Output	Primary error risk
Collect / source	License-compatible aerial/oblique images	Select scenes resembling actual camera, altitude, scale, occlusion and disaster context	Raw image corpus	Domain shift; licensing problems
Taxonomy	Raw corpus	Label exactly person, vehicle, debris for baseline	Class definitions + labels	Ambiguous labels; class leakage
Scene-level split	Labeled corpus	Split train/val/test by scene/location where possible	Non-leaky datasets	Near-duplicate leakage
Augment	Train images	Scale, crop, brightness/contrast, blur, mild rotation, sensor noise	Augmented minibatches	Unrealistic augmentation
Train	Train set	YOLO11n optimization	PyTorch checkpoint	Overfit/underfit
Validate	Val set	Precision/recall/mAP + failure inspection	Model candidate	Threshold overfitting
Calibrate	Representative production images	INT8 Hailo calibration	Calibration set / stats	Domain mismatch
Export	YOLO checkpoint + calibration + DFC	Hailo HEF compile	model.hef + metadata	Quantization loss
Deploy test	HEF + exact Pi/HailoRT	End-to-end FPS, latency, thermal soak	Release build	Host/device mismatch

4.2 Loss and optimization terminology

For YOLO11 training, the implementation exposes weighted detection loss components including box localization, classification and Distribution Focal Loss (DFL). A useful documentation-level representation is:

$$L_{\text{total}} = \lambda_{\text{box}} * L_{\text{box}} + \lambda_{\text{cls}} * L_{\text{cls}} + \lambda_{\text{dfl}} * L_{\text{dfl}}$$

The current Ultralytics training documentation lists optimizer choices including SGD, Adam, AdamW, NAdam, RAdam, RMSProp and MuSGD, with optimizer=auto able to select a training optimizer based on iteration count. Therefore the AERIS release process should record the resolved optimizer and all training hyperparameters in a training manifest instead of assuming that a generic checkpoint implies a fixed optimizer.

Recommended reproducibility rule

For the first research baseline, allow the library auto-selection if that is how the baseline is trained, but capture the resolved optimizer, learning-rate settings, epochs, batch size, seed, image size, augmentation settings and library commit/version in training_manifest.yaml. If the team later needs deterministic comparison across experiments, explicitly freeze one optimizer (for example AdamW) and record the exact hyperparameters. This is an engineering recommendation, not a Rev 2.0 frozen value.

4.3 Training errors and what they mean

Error / signal	What it indicates	Action
High training accuracy, poor validation	Overfitting or scene leakage	Check scene split; reduce augmentation mismatch; add representative scenes
High recall, low precision	Too many false positives	Improve labels; hard-negative mining; threshold/NMS tuning; calibration review
High precision, low recall	Missed targets	Improve data diversity; inspect small/occluded targets; test image size and class balance
PyTorch model good, HEF worse	Quantization/calibration loss or preprocessing mismatch	Use representative calibration data; ensure identical preprocessing; compare on same test split
Good mAP, poor field performance	Domain shift	Collect field-like images; validate by location/scene; retrain release model
Low FPS / high latency	Model or pipeline too expensive	Measure Hailo-only vs end-to-end latency; reduce resolution/model size; remove queue backlog
Event storm despite good detector	Tracking/confirmation/de-dup issue	Inspect track IDs, time windows, spatial suppression and cooldown

4.4 Release metrics

Metric	Definition / use	Release interpretation
Precision	$TP / (TP + FP)$	Measures false-positive burden
Recall	$TP / (TP + FN)$	Measures missed detections
mAP50	Mean AP at IoU 0.50	Standard detector reporting
mAP50-95	Average AP across IoU thresholds	Stricter localization quality
Inference p50/p95	Distribution of inference latency	Used to choose safe search speed
End-to-end event latency	Capture -> dashboard incident time	Operational responsiveness
Geo median / P95 error	Distance from estimated to surveyed target	Required to bound mission use
False alerts/run	Incorrect or duplicate incident count	Operational usefulness
Telemetry freshness	Age of vehicle state used in decisions	Reject stale pose/commands
Log completeness	Expected vs persisted records	Auditability

5. Mathematics and Algorithms Reference

5.1 Coordinate systems and camera projection

The geo-tagging implementation uses the standard pinhole-camera transformation with explicit frame rotations. The purpose is to map an image point into a 3D ray, then intersect that ray with an assumed ground plane.

```
p = [u, v, 1]^T
K = [[fx, 0, cx], [0, fy, cy], [0, 0, 1]]
r_c = K^-1 p
r_w = R_bw R_cb r_c
lambda = (z_ground - C_z) / r_w,z
P_target = C + lambda r_w
```

- K contains focal lengths and principal point; it is estimated by camera calibration.
- R_cb is the fixed camera-to-body transform from the physical mount.
- R_bw is the vehicle/body-to-world pose from flight telemetry.
- C is the camera position in the world frame, not merely the aircraft GPS point.
- z_ground is an estimated local ground height; uneven terrain therefore contributes error.

5.2 Bounding-box and overlap mathematics

$$\text{IoU}(B1, B2) = \text{Area}(B1 \text{ intersect } B2) / \text{Area}(B1 \text{ union } B2)$$

IoU is used by detector postprocessing to reason about overlap and by evaluation to measure localization quality.

5.3 Temporal association

```
t_vehicle* = argmin_t |t_vehicle - t_detection|
accept only if |t_vehicle* - t_detection| <= Delta_t_sync
```

This nearest-neighbor time association is intentionally simple and deterministic. It becomes unsafe when clocks, timestamps or telemetry freshness are unbounded, so the tolerance and age are part of the data contract.

5.4 Filtering

$$d_{\text{filtered}}(k) = \text{median}[d(k-2), d(k-1), d(k), d(k+1), d(k+2)]$$

The median filter is used for TFmini distance because it rejects short impulsive spikes better than a plain mean. The selected 5-sample window is the Rev 2.0 baseline.

5.5 Coverage planning

```
W = 2*h*tan(theta/2)
S = W*(1-overlap)
```

W is the approximate horizontal camera footprint at altitude h, theta is the horizontal field of view and overlap is the planning overlap. Rev 2.0 starts with 20% overlap and a controlled demonstration speed of

2.5-3.0 m/s. The route generator clips parallel sweep lines to the mission polygon and alternates lane direction.

5.6 Quadrotor dynamics model for validation / digital twin

$$\begin{aligned} m \cdot \dot{v} &= m \cdot g + R \cdot f_b \\ \dot{R} &= R \cdot \text{skew}(\Omega) \\ J \cdot \dot{\Omega} + \Omega \times (J \cdot \Omega) &= \tau \end{aligned}$$

These equations describe the rigid-body dynamics used to reason about thrust margin and digital-twin parameters. They are not an instruction to replace ArduPilot with a custom flight controller.

5.7 Center of gravity and engineering checks

$$\begin{aligned} CG_x &= \sum(m_i \cdot x_i) / \sum(m_i) \\ CG_y &= \sum(m_i \cdot y_i) / \sum(m_i) \\ CG_z &= \sum(m_i \cdot z_i) / \sum(m_i) \end{aligned}$$

5.8 Geo-error budgeting

Rev 2.0 gives an illustrative root-sum-square calculation using assumed independent components (GPS, centroid/camera, attitude projection and altitude/ground), yielding approximately 2.4 m. The document explicitly states this is not a guarantee. The actual acceptance criterion should be based on surveyed markers and a measured 95th percentile error.

6. Algorithm / Logic Selection Matrix

Function	Selected method	Why	Inputs	Outputs	Frozen or tunable
Object detection	Custom YOLO11n	Small edge baseline; Hailo-8L compatible export path	640x640 tensor	Boxes/classes/confidence	Model architecture frozen; weights trained
Detector decoding / NMS	IoU-based NMS	Standard suppression for duplicate boxes	Raw detector output	DetectionBatch	Baseline thresholds tunable
Tracking	ByteTrack	Stable IDs; supports temporal confirmation and dedup	DetectionBatch + track history	TrackBatch	Algorithm frozen
Confirmation	3-of-5 persistence	Simple, interpretable temporal filter	Track history	ConfirmedTrack	Window/threshold tunable
Priority	Weighted score	Transparent, auditable; no black-box triage	c,t,s,m	PriorityDecision	Weights tunable
Geo-tagging	Pinhole camera + pose + ground plane	Uses onboard pose and calibrated camera	Pixel + K + pose	GeoEstimate	Calibration/ground model tunable
Obstacle filter	5-sample median	Suppresses impulse spikes	TFmini samples	Filtered range	Window/threshold tunable
Obstacle response	NORMAL/ WARNING/HOLD FSM	Conservative with single forward sensor	Filtered range + flight state	ObstacleState	Thresholds tunable
Search planning	Polygon clipping + lawnmower sweep	Deterministic coverage pattern	Polygon + altitude + FOV	Waypoints	Overlap/speed tunable

Function	Selected method	Why	Inputs	Outputs	Frozen or tunable
Mission control	Finite state machine	Auditable deterministic behavior	Mission + vehicle + incident + operator	MissionCommand	State guards configurable
Flight control	ArduPilot built-in controllers	Separates safety-critical flight control from AI	Sensors + modes + mission	Actuator outputs	ArduPilot parameters tuned
Communications	MAVLink / SiK / Wi-Fi / ELRS	Different channels for flight, video/events and manual override	Telemetry/video/RC	Commands + data	Port details must be verified on hardware

7. Data Contracts and Interfaces

7.1 Core typed runtime packets

FramePacket

```

frame_id: int
t_capture_utc: str
image: array
width: int
height: int
camera_config_id: str

```

Detection

```

class_name: str
confidence: float
bbox_xyxy: tuple[float,float,float,float]
frame_id: int
timestamp_utc: str

```

Track

```

track_id: int
class_name: str
confidence: float
bbox_xyxy: tuple
last_seen_utc: str
age_frames: int

```

VehicleState

```

timestamp_utc: str
lat: float
lon: float
altitude_m: float
roll_rad: float
pitch_rad: float
yaw_rad: float
mode: str
gps_ok: bool
ekf_ok: bool

```

battery_remaining: float
 heartbeat_age_ms: float

ObstacleState

timestamp_utc: str
 distance_m: float
 quality: str
 state: NORMAL | WARNING | HOLD

IncidentEvent

mission_id: str
 event_id: str
 timestamp_utc: str
 class: str
 confidence: float
 track_id: int
 drone_lat: float
 drone_lon: float
 target_lat: float
 target_lon: float
 altitude_m: float
 priority: str
 geo_quality: str
 flight_mode: str
 operator_status: str

MissionCommand

mission_id: str
 command_type: str
 arguments: dict
 reason: str
 issued_at_utc: str
 allowed: bool

7.2 Canonical event JSON

```
{
  "mission_id": "M-2026-001",
  "event_id": "INC-0001",
  "timestamp_utc": "...",
  "class": "person",
  "confidence": 0.87,
  "track_id": 12,
  "drone_lat": 0.0,
  "drone_lon": 0.0,
  "target_lat": 0.0,
  "target_lon": 0.0,
  "altitude_m": 8.0,
  "priority": "HIGH",
  "geo_quality": "APPROXIMATE",
  "flight_mode": "AUTO",
  "operator_status": "UNVERIFIED"
}
```

7.3 Interface ownership

Interface	Producer	Consumer	Data	Rule
CSI camera	Camera module	Capture worker	Frames	Latest-frame policy; timestamp every frame
Hailo runtime	Inference worker	AI pipeline	Tensor -> detections	Exact HEF preprocessing contract
Vehicle telemetry	Pixhawk/MAVLink	Telemetry task	GPS, attitude, battery, mode, health	Timestamp + freshness required
TFmini UART	TFmini Plus	Sensor task	Range samples	Validate range + timestamps
Mission command	Mission Manager	MAVLink layer	Allow-listed commands	No raw servo API from dashboard
Event stream	Event Manager	WebSocket server	IncidentEvent	JSON schema versioned
Operator decision	Dashboard	Mission Manager	Verify/authorize/ignore	Explicit audit record
Flight log	ArduPilot	Post-flight tools	Controller log	Separate source-of-truth artifact

8. Runtime Concurrency and Process Architecture

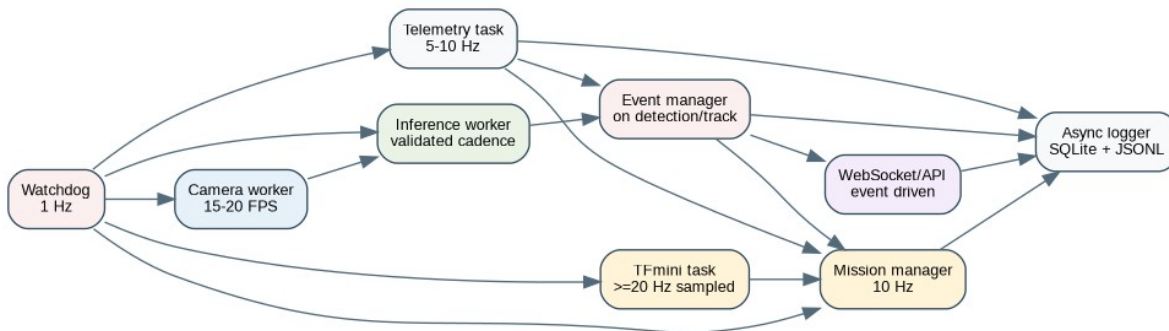


Figure 4. Companion-computer runtime processes and cadences.

Runtime component	Cadence / trigger	Inputs	Outputs	Reason for separation
Camera worker	15-20 FPS target	Camera	FramePacket ring buffer	Keeps capture responsive
Inference worker	As fast as validated	FramePacket	DetectionBatch	Keeps NPU workload isolated
Telemetry task	5-10 Hz	MAVLink	VehicleState cache	Provides fresh pose/state
TFmini task	Sensor rate / >=20 Hz sampled	UART range	ObstacleState	Safety path stays current
Event manager	On detection/track update	Detection + track + state	IncidentEvent	Contains AI-to-event logic
Mission manager	10 Hz	Mission + incident + safety + operator	MissionCommand	Deterministic central decision point
GCS/Web server	Event driven	Events/telemetry/video	Dashboard updates	UI should not block autonomy

Runtime component	Cadence / trigger	Inputs	Outputs	Reason for separation
Logger	Buffered async	All critical packets	SQLite + JSONL	Avoid disk I/O blocking inference/mission
Watchdog	1 Hz	Process heartbeats/temp	Health actions	Detect hung tasks

Queue/backpressure rule

Avoid unbounded queues. For perception, stale frames are less useful than fresh frames, so the buffer should be bounded and allowed to drop old frames. For mission commands and incident events, preserve ordering and auditability; use bounded retry/error handling rather than silent loss.

9. Safety, Errors, Failsafes and Degraded Modes

Failure	How detected	Immediate software behavior	Who owns final safety action	What is never allowed
Pi process crash	Watchdog / missing heartbeat	AI unavailable; mission policy abort/return	Pixhawk for flight	AI crash must not cause uncontrolled actuation
Hailo failure	Inference heartbeat	Stop AI events; mark AI unhealthy	Mission policy + Pixhawk	Claiming AI healthy when inference is stale
Camera failure	Capture health	Disable perception events	Pixhawk	Using stale image as current observation
MAVLink stale	Heartbeat age	Close command gate; abort mission action	Pixhawk	Sending commands on stale state
GPS/EKF unhealthy	VehicleState health	No autonomous response; transition to safe state	Pixhawk	Geo-driven response on invalid pose
TFmini invalid/stale	Quality/age check	Conservative obstacle state	Mission + Pixhawk	Claiming 360-degree avoidance
Battery low	ArduPilot battery failsafe	Pi stops response/payload actions	ArduPilot	Payload release during low battery
RC loss	ArduPilot RC failsafe	Controller executes configured action	ArduPilot	Pi attempting to override stronger failsafe
GCS/SiK loss	GCS heartbeat / link state	Controller continues configured failsafe	ArduPilot	Treating Wi-Fi loss as total aircraft loss if SiK/RC remain healthy
Duplicate AI alert	track/cooldown/spatial cell	Suppress repeat incident	Event Manager	Alert storm
Invalid geo geometry	ray/pose validation	Fallback to aircraft GPS labeled approximate	Mission Manager	High-confidence target navigation from invalid projection
Servo jam	Release feedback / state mismatch	Abort release and return/hold	Mission Manager + operator	Blindly repeating release

9.1 Safety precedence

CRITICAL VEHICLE SAFETY > RC OVERRIDE > FLIGHT CONTROLLER FAILSAFE > MISSION ABORT > AI DECISION

This ordering is a software ownership rule: higher levels are not permitted to veto stronger lower-level safety mechanisms.

10. Full Software Architecture - Final Form

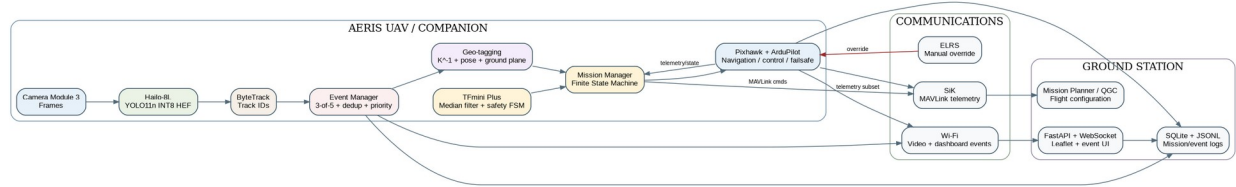


Figure 5. Final AERIS architecture with explicit software ownership boundaries.

10.1 Layer ownership

Layer	Owner	Responsibilities	Must not do
Perception	Pi + Hailo	Capture, detection, tracking, confirmation	Direct motor/servo actuation
Localization	Pi	Camera model + pose sync + geo projection	Pretend GPS of drone equals GPS of target
Decision	Pi Mission Manager	Priority, state transitions, interlocks	Override flight-controller failsafe
Flight control	Pixhawk/ArduPilot	Attitude, navigation, modes, failsafe, actuation	Depend on Pi process health for stabilization
Communication	MAVLink/SiK/Wi-Fi/ELRS	Telemetry, events/video, manual control	Share interfaces ambiguously
Ground UI	Mission Planner/QGC + FastAPI dashboard	Planning, monitoring, operator authorization	Direct raw actuator endpoint
Persistence	SQLite/JSONL + Pixhawk logs	Audit, replay, metrics	Overwrite prior mission artifacts

10.2 Communication paths

Path	Physical / logical link	Purpose	Primary data
Companion link	Pixhawk <-> Pi over dedicated serial/USB-UART	Mission supervision + telemetry	MAVLink
Telemetry link	Pixhawk <-> SiK <-> GCS	Ground flight telemetry / commands	MAVLink
Video/event link	Pi <-> ground over local Wi-Fi	Live video + AI dashboard	MJPEG/RTP/WebRTC + WebSocket/HTTP
Manual override	ELRS receiver <-> Pixhawk	Pilot/manual safety control	RC/CRSF path as configured

11. Software Repository - What We Will Create

```

aeris/
+-- firmware/
|   +-- ardupilot/          # firmware/version notes
|   +-- parameters/
|       +-- prototype.param  # immutable flight parameter release artifact
+-- companion/
|   +-- vision/
|       |   +-- capture.py    # FramePacket producer
|       |   +-- preprocess.py # model input preparation
|       |   +-- infer.py      # Hailo/YOLO inference adapter
|       |   +-- tracker.py    # ByteTrack adapter
|   +-- sensors/
|       |   +-- tfmini.py     # range reader + filter
|       |   +-- telemetry.py  # MAVLink telemetry cache
|   +-- geo/
|       |   +-- camera_model.py # K, distortion, calibration
|       |   +-- projection.py   # ray -> ground intersection
|   +-- mission/
|       |   +-- state_machine.py # deterministic mission FSM
|       |   +-- coverage.py      # polygon -> lawnmower route
|       |   +-- response.py      # response/release guards
|   +-- events/
|       |   +-- event_manager.py # detection -> incident pipeline
|       |   +-- priority.py      # weighted score
|       |   +-- schemas.py       # typed packets / validation
|   +-- comms/
|       |   +-- mavlink.py      # allow-listed commands + ACKs
|       |   +-- websocket.py    # dashboard push
|       |   +-- video.py        # video transport adapter
|   +-- logging/
|       |   +-- db.py           # SQLite
|       |   +-- jsonl.py        # append-only event log
|   +-- health/
|       |   +-- watchdog.py     # process/temperature health
|   +-- config/
|       |   +-- loader.py       # validated config loading
|   +-- main.py                 # process supervisor / startup
+-- dashboard/
|   +-- app.py                  # FastAPI app
|   +-- static/                 # JS/CSS/assets
|   +-- templates/              # HTML views
+-- models/
|   +-- yolo11n_aeris_hailo/
|       +-- model.hef           # release inference artifact
|       +-- metadata.yaml
|       +-- classes.txt
+-- configs/
|   +-- mission.yaml
|   +-- ai.yaml
|   +-- sensor.yaml
|   +-- network.yaml
+-- training/

```

```

| +-- dataset.yaml
| +-- train.py
| +-- eval.py
| +-- export_hailo.py
| +-- training_manifest.yaml
+-- sim/
| +-- sitl/           # ArduPilot SITL scenarios
| +-- gazebo/         # world/models/sensors
| +-- replay/         # event/telemetry replay
+-- tests/
| +-- unit/
| +-- integration/
| +-- hardware/
| +-- fixtures/
+-- cad/
| +-- frame_mounts/
| +-- payload/
+-- docs/
| +-- architecture/
| +-- pinmap.csv
| +-- test_reports/
| +-- release_notes/
+-- pyproject.toml / requirements.lock
+-- README.md
+-- Makefile or task runner
+-- .gitignore

```

11.1 Why this folder structure

Directory	Why it exists	Design benefit
companion/vision	Perception only	Easy to benchmark camera/AI independently
companion/sensors	Hardware input adapters	Hardware changes stay localized
companion/geo	Calibration and geometry	Geo math is testable without flight
companion/mission	Autonomy state and route logic	Mission behavior is deterministic and replayable
companion/events	Event semantics	One canonical incident pipeline
comms	External interfaces	Transport details separated from business logic
logging	Persistence	I/O is isolated from real-time logic
health	Watchdog and health	Degraded states are first-class
models	Deployment artifacts	Exact HEF and metadata travel with release
configs	Runtime tuning	No magic numbers buried in code
training	Model lifecycle	Training/export/evaluation reproducible
sim	SITL/Gazebo/replay	Safety-first validation before hardware
tests	Acceptance evidence	Unit/integration/hardware checks organized by layer

Directory	Why it exists	Design benefit
docs	Engineering evidence	Pin maps, reports, decisions and releases retained

Creation rule

Every module should have one primary responsibility, explicit inputs/outputs and unit tests against deterministic fixtures. Do not let dashboard code import GPIO/PWM logic or let the detector call servo functions directly. This structure enforces the safety boundary by dependency direction.

12. Internal APIs / Module Boundaries

Module	Primary functions / methods	Inputs	Outputs
capture.py	start(), read_latest(), health()	Camera config	FramePacket
preprocess.py	prepare(frame), map_box_to_source()	FramePacket	TensorFrame + PreprocessMeta
infer.py	load_model(), infer(tensor)	TensorFrame	DetectionBatch
tracker.py	update(detections)	DetectionBatch	TrackBatch
event_manager.py	update_tracks(), confirm(), build_event()	TrackBatch + VehicleState + GeoEstimate	IncidentEvent
priority.py	score(event_context)	c,t,s,m	PriorityDecision
telemetry.py	connect(), poll(), get_state()	MAVLink stream	VehicleState
tfmini.py	read(), filter(), state()	UART samples	ObstacleState
projection.py	project(pixel, pose, calib)	pixel + pose + calibration	GeoEstimate
coverage.py	plan_polygon()	polygon + altitude + FOV + overlap	WaypointMission
state_machine.py	step(context)	mission + vehicle + events + safety	MissionCommand / State
mavlink.py	send_allowed(), heartbeat(), ack()	MissionCommand	CommandAck
response.py	can_release(context)	operator + safety + payload	bool + reason
db.py	insert_event(), mission_start(), mission_end()	schemas	SQLite transaction result
websocket.py	publish_event(), publish_state()	events/state	client messages
watchdog.py	register(), heartbeat(), evaluate()	process heartbeats	health actions

13. How We Will Actually Create the Software

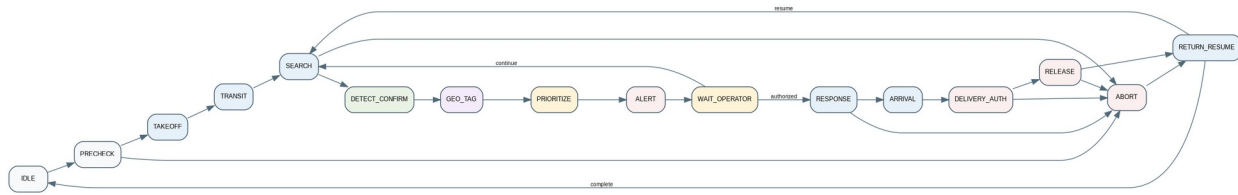


Figure 6. Final mission state machine.

13.1 Build the contract first

Create schemas.py and the sample JSON fixtures. Freeze field names, timestamps, IDs and status enums before connecting hardware.

13.2 Build offline perception

Run camera/video-file input -> preprocessing -> YOLO11n -> NMS -> ByteTrack -> confirmation using recorded datasets. No flight controller yet.

13.3 Train and benchmark

Build the dataset pipeline, train/evaluate the custom model, inspect precision/recall/mAP, export a Hailo HEF, benchmark the exact Pi/Hailo runtime.

13.4 Build geo-tagging as a pure library

Calibrate the camera and implement projection with unit tests on synthetic and surveyed points. No flight command dependency.

13.5 Build telemetry adapter

Connect Pi to Pixhawk in bench/SITL, create VehicleState cache, freshness rules and command allow-list. No autonomous flight command first.

13.6 Build mission FSM in SITL

Feed synthetic detections, telemetry, operator approvals and faults into the state machine. Verify all transitions and abort paths.

13.7 Build dashboard

Subscribe to event/state streams and create operator review/authorization. Verify that UI restart does not corrupt mission logic.

13.8 Integrate obstacle sensor

Add TFmini filter and safety states; test threshold crossing with controlled targets before allowing it to influence missions.

13.9 Integrate payload only after flight safety

Bench-test servo/latch and authorization logic independently. Keep the payload path physically disconnected during earlier flight tests.

13.10 Run integrated controlled flight tests

Manual/assisted -> GPS/RTL/failsafe -> search -> perception -> event -> response -> return/resume, adding one subsystem at a time.

13.11 Freeze the release

Store firmware version, parameter file, HEF checksum, Python lock, configs, test reports and exact source commit in a single release manifest.

14. Implementation Roadmap and Exit Criteria

Phase	Work package	What we create	Exit criterion
S0	Specification freeze	schemas, config files, version manifest, architecture decisions	All interfaces documented
S1	Repository bootstrap	folder tree, CI/test runner, logging primitives	Unit tests execute
S2	Offline camera pipeline	capture + preprocess + replay source	Stable FramePacket flow
S3	Model training	dataset + training + evaluation + release checkpoint	Measured validation report
S4	Hailo deployment	HEF export + HailoRT inference adapter	Stable bench inference / thermal soak
S5	Tracking/event layer	ByteTrack + confirmation + de-dup + schema	Repeatable event generation
S6	Geo library	calibration + projection + surveyed test harness	Measured geo-error report
S7	MAVLink layer	telemetry cache + allow-listed commands + watchdog	SITL + bench link pass
S8	Mission FSM	state_machine + coverage + response guards	All transition tests pass
S9	Dashboard	FastAPI + WebSocket + map + event UI	Operator view and authorization pass
S10	TFmini safety	filter + NORMAL/WARNING/HOLD	Controlled obstacle test pass
S11	Integrated SITL	full loop with simulated faults	Scenario suite passes
S12	Physical integration	controlled hardware wiring + bench checks	Hardware acceptance gates pass
S13	Flight integration	manual/assist + RTL/failsafe + autonomous search	Controlled mission acceptance
S14	Release hardening	logs, scripts, manifests, backups, demo runbook	Three representative end-to-end runs with no critical regression

15. Test Strategy - What Proves Each Claim

Layer	Test	Input	Pass evidence
Schemas	Round-trip / invalid field tests	Valid + invalid JSON/objects	Correct validation and error reasons
Camera	Replay capture test	Recorded images/video	Expected frame rate/timestamps
AI	Held-out scene test	Test dataset	Precision/recall/mAP report
Hailo	Parity test	Same images in PyTorch and HEF	Measured quantization/deployment gap
Tracking	Annotated sequence	Known trajectories	Stable IDs / acceptable association
Event	Synthetic frame sequence	1-frame, intermittent, persistent detections	Correct 3-of-5 and dedup behavior
Geo	Surveyed markers	Known coordinates + image observations	Median and P95 geo error
MAVLink	Heartbeat / command tests	SITL + bench link	Correct state cache + command ACKs
FSM	State transition tests	Synthetic events/faults	No illegal transitions
TFmini	Known target distances	Controlled obstacles	Distance and state thresholds validated
Dashboard	UI restart / network loss	Event stream	No silent loss; safe degraded state
Failsafe	RC/GCS/battery/GPS faults	Controlled test conditions	Configured controller action occurs
Payload	Bench authorization test	Operator + fault combinations	Zero unauthorized releases
Integrated	End-to-end mission	Search scenario	Search -> event -> review -> optional response -> return

15.1 The acceptance gate ordering

POWER -> PROPULSION -> FLIGHT -> COMPANION -> AI -> MAVLink -> GCS -> EVENTS -> GEO -> SEARCH
-> OBSTACLE -> PAYLOAD -> INTEGRATED MISSION -> DIGITAL TWIN -> DEMO HARDENING

Do not reverse safety order

Do not attach the payload or begin autonomous search flights before stable manual flight, GPS, return-to-launch and battery-failsafe tests are complete.

16. Initial Parameter Ledger - Frozen vs Tunable

Parameter	Baseline	Status	Where stored	How tuned / validated
Model	Custom YOLO11n	Frozen baseline	models/ + training manifest	Weights trained on project dataset
Input size	640x640	Frozen baseline	ai.yaml + HEF metadata	Change only with new HEF/export

Parameter	Baseline	Status	Where stored	How tuned / validated
Classes	person, vehicle, debris	Frozen baseline	classes.txt	Change only with retraining/evaluation
Detector confidence	0.35	Starting value	ai.yaml	Tune on held-out/field set
Event confidence	0.60	Starting value	ai.yaml	Tune on event precision/recall
NMS IoU	0.45	Starting value	ai.yaml	Tune on detector overlap behavior
Confirmation	3 of last 5	Starting value	ai.yaml	Tune from recorded sequences
Cooldown	10 s + spatial cell	Starting value	ai.yaml	Tune duplicate rate
Priority weights	0.55/0.20/0.15/0.10	Starting baseline	ai.yaml	Tune with mission trace review
TFmini states	8.0m / 6.5m	Starting baseline	sensor.yaml	Tune from measured response
Search overlap	20%	Starting baseline	mission.yaml	Tune coverage/energy tradeoff
Search speed	2.5-3.0 m/s	Starting demo range	mission.yaml	Tune from detection quality and braking
Battery failsafes	Initial Rev 2.0 values	Starting baseline	ArduPilot param file	Validate against actual battery sag and firmware
Sync tolerance	Delta_t_sync	Must be explicitly frozen before field release	mission/sensor config	Set from timestamp alignment test
Inference FPS	15-20 target	Measured target	test report	Measure exact HEF + pipeline
Geo error	No fixed claim	Measured only	test report	Surveyed markers; report median/P95

17. Version Freeze and Reproducibility

Artifact	What to freeze	Why
ArduPilot	Exact firmware build/version + full parameter file	Controller behavior can change with firmware
Raspberry Pi OS	Exact image/release identifier	OS package/runtime reproducibility
Python	Exact major/minor + lock file	Dependency reproducibility
Ultralytics	Exact package version / commit	Model graph and export behavior
HailoRT	Exact runtime version	HEF compatibility/performance
HEF	model.hef + metadata.yaml + checksum	Device deployment artifact
Dataset	Dataset version + source licenses + label taxonomy	Training reproducibility
Training	Seed, optimizer, learning rate, epochs, batch size, augmentation, config	Experiment reproducibility

Artifact	What to freeze	Why
Mission config	mission.yaml / sensor.yaml / ai.yaml / network.yaml	Field behavior reproducibility
Source	Git commit tag	Exact software build
Test evidence	Reports + logs + scenario IDs	Claim traceability

```

release_manifest.yaml
project: AERIS
source_commit: <git-sha>
firmware: <ardupilot-version>
python: <version>
ultralytics: <version>
hailort: <version>
hef_sha256: <hash>
dataset_version: <id>
training_manifest: training_manifest.yaml
configs: [mission.yaml, ai.yaml, sensor.yaml, network.yaml]
validation_report: docs/test_reports/<release>.md

```

18. Current Toolchain Verification Notes

The project baseline is preserved, but the following implementation notes were checked against current official documentation during preparation of this document:

- Ultralytics currently documents Hailo export for YOLO11 to Hailo-8/Hailo-8L, with fixed input shape and INT8 Hailo export. The current guide recommends representative calibration data and notes 1,024 images as a best-accuracy starting point. The exact HEF must be validated on the exact HailoRT/runtime stack used on the prototype.
- Ultralytics current training documentation exposes optimizer choices including AdamW and MuSGD and allows optimizer=auto; therefore the final training manifest must record the resolved optimizer rather than relying on an undocumented assumption.
- ArduPilot documents companion computers as MAVLink-connected Linux systems that receive autopilot telemetry and can make higher-level intelligent decisions. This matches the AERIS boundary between Pi mission/perception logic and Pixhawk flight control.
- ArduPilot GCS failsafe documentation describes heartbeat-based loss detection with configurable failsafe actions. This reinforces the rule that stronger flight-controller failsafes must remain independent of the Pi application.

19. What Is Not Yet Frozen / Must Be Measured

Item	Why unresolved	Required action before final claim
Exact all-up mass	Depends on purchased parts, wiring, mounts, battery and payload	Weigh final aircraft
Actual thrust	Depends on motor/prop/battery combination	Bench thrust test / measured margin

Item	Why unresolved	Required action before final claim
Actual hover power / flight time	Mission, mass and battery sag dependent	Repeatable logged flight test
Actual AI precision/recall	Depends on trained dataset and scene	Held-out + field evaluation
Actual HEF FPS/latency	Depends on exact export/runtime	Bench benchmark on exact Pi/Hailo
Actual geo-tag error	Depends on calibration, pose, ground model and GNSS	Surveyed-marker calibration
Actual obstacle stopping distance	Depends on mass, airspeed, controller response and surface	Controlled braking/hold test
Exact Pixhawk pinout	Board revisions/clones can differ	Compare purchased board manual before wiring
Exact ELRS receiver/CRSF mapping	Depends on selected receiver firmware/hardware	Freeze receiver, firmware and wiring
Legal RF configuration	Depends on location/event rules	Confirm site-appropriate radio configuration

20. Final Architecture and What We Are Going to Build

The final product is a modular companion-computer software stack running on the Raspberry Pi 5, connected to the Pixhawk over MAVLink, accelerated by the Hailo-8L, and connected to the ground through the defined telemetry/video/event channels. The software is deliberately built so that camera/replay inputs can substitute for the physical camera during development and so that the mission/event logic can be tested in SITL before flight hardware is involved.

20.1 Final working in one chain

MISSION CONFIG

- > PRECHECK
- > TAKEOFF / TRANSIT (ArduPilot owns flight)
- > SEARCH (lawnmower waypoints)
- > CAMERA FRAME
- > PREPROCESS
- > YOLO11n @ HAILO-8L
- > NMS / DETECTION
- > BYTETRACK
- > 3/5 CONFIRMATION + DEDUP
- > TELEMETRY TIMESTAMP ASSOCIATION
- > CAMERA GEOMETRY GEO-TAG
- > PRIORITY SCORE
- > INCIDENT LOG + DASHBOARD
- > HUMAN REVIEW
- > OPTIONAL RESPONSE AUTHORIZATION
- > MISSION FSM
- > MAVLINK ALLOW-LIST COMMAND
- > PIXHAWK / ARDUPILOT
- > ARRIVAL / PAYLOAD INTERLOCK
- > OPTIONAL HUMAN-AUTHORIZED RELEASE
- > RETURN / RESUME
- > POST-FLIGHT LOG + REPLAY

20.2 What each major software package becomes

Package	Final role
vision	Real-time perception adapter; hardware-independent input contract
events	Turns detections into incidents and priority decisions
geo	Calibration and target-coordinate estimation
mission	Mission FSM and deterministic route/response decisions
sensors	Vehicle and obstacle state acquisition
comms	MAVLink, dashboard and video transport adapters
logging	Persistent mission/event artifacts
health	Process and thermal watchdog
dashboard	Human visualization and authorization
training	Dataset/model lifecycle and release artifacts
sim	SITL/Gazebo/replay environment
tests	Proof that each claim and boundary works

20.3 Build principle

We will build the software from the inside out: first freeze the data contracts, then make the perception pipeline work on a laptop using recorded data, then move the same pipeline onto Hailo, then add geo-tagging and telemetry, then the mission FSM and dashboard, then SITL, and only after those gates pass do we connect the complete stack to the aircraft. This approach keeps the software testable and prevents hardware availability from becoming the bottleneck for AI and mission development.

21. Mentor Review Checklist

- Is the model choice appropriate for the target Hailo-8L latency/accuracy requirement?
- Are the training classes and dataset representative of the actual camera view?
- Are the detector confidence, NMS IoU and 3-of-5 confirmation thresholds experimentally justified?
- Is ByteTrack sufficient for the intended event de-duplication, and what tracker metric should be reported?
- Is the camera calibration / ground-plane assumption valid for the chosen camera mount and altitude range?
- Is the temporal synchronization tolerance small enough and measurable?
- Do the weighted priority coefficients reflect the intended operational policy and can they be audited?
- Are the TFmini thresholds conservative enough for the measured vehicle response?
- Is the mission FSM complete for all normal and failure paths?
- Are command allow-lists and safety ownership boundaries implemented in code rather than only documented?
- What additional sensors or models would be justified after baseline validation?
- Which empirical performance targets should be frozen after the first controlled experiments?

22. Source and Technical References

- [S1] AERIS Prototype Master Build Specification - Revision 2.0, supplied project document, 26 September 2026. Primary project baseline for architecture, stack, model, repository, mission states, validation gates and known limits.
- [S2] AERIS Concept & Prototype Document, supplied project document.
- [S3] AERIS / GUJCOST / Robofest technical application and Final Hackathon Build Plan, supplied project materials.
- [W1] Ultralytics, Hailo Export for Ultralytics YOLO Models. <https://docs.ultralytics.com/integrations/hailo>
- [W2] Ultralytics, Model Training with Ultralytics YOLO. <https://docs.ultralytics.com/modes/train>
- [W3] Ultralytics, YOLO Architecture Explained. <https://docs.ultralytics.com/guides/yolo-architecture>
- [W4] ArduPilot, Companion Computers. <https://ardupilot.org/copter/docs/common-companion-computers.html>
- [W5] ArduPilot, GCS Failsafe. <https://en.ardupilot.org/copter/docs/gcs-failsafe.html>
- [R1] Redmon et al., You Only Look Once: Unified, Real-Time Object Detection.
- [R2] Zhang et al., ByteTrack: Multi-Object Tracking by Associating Every Detection Box.
- [R3] Coverage path planning literature cited in S1.

23. Final Engineering Statement

Build readiness

The software architecture is sufficiently explicit to begin implementation: every major stage has a defined input, processing method, output and destination; the model/algorithm/mathematical method is stated; runtime processes are separated; repository ownership is defined; and validation gates are ordered. Remaining uncertainty is primarily empirical (model performance, exact timing, geo-error, stopping response, hardware-specific pinout and final physical mass/thrust), not an unspecified software architecture.